

Phase 2 - final SQL queries

Data Vortex Round 1 Phase 2 · every query behind the E/M/H answers.

Team	Forge-X
Team head	Kunal Choudhary
Covers	E1-E5, M1-M5, H1-H6 (21 challenges) + 17 verification queries
Engine	SQLite, CTEs and window functions
Source	output/social_engine.db, built from the recovered intake files
Reproduce	python src/build_db.py && python src/run_sql.py

Slot 1 of 4 - SQL query

Every figure in this document was produced by executing the queries against the recovered database in this run, and is read back out of output/phase2_sql_results.json. Nothing was transcribed by hand.

How to read the query file

- **Engagement** is written `COALESCE(likes,0)+COALESCE(shares,0)+COALESCE(comments,0)` everywhere. SQL's bare `+` returns `NULL` when any operand is `NULL`, which would silently drop 1,532 rows (15%) from every aggregate instead of raising an error.
- **Missing values are never imputed.** Where a challenge says to ignore posts with no like count (E2), or where a comparison is undefined without it (M5, H6), the query filters them out rather than reading a blank as zero.
- **The 'Unspecified' platform is a data gap** (1,541 posts, 15.08%), not a platform. It is kept for volume questions and excluded wherever a per-platform rate or a winner is computed.
- **Every LIMIT carries a deterministic tie-break**, so a top-10 list cannot change between runs.
- **Each challenge is followed by its verification queries** (E1b, M5b, H4b ...). They test the headline answer against the assumptions it rests on, and some of them overturn it. They are part of the answer, not an appendix.

EASY questions

E1 - Platform Popularity

Challenge. Determine which social media platform has the highest number of posts. Ignore posts where the platform is missing. Return the platform name and number of posts.

```
SELECT platform,
       COUNT(*) AS n_posts
FROM posts
WHERE platform <> 'Unspecified'
GROUP BY platform
ORDER BY n_posts DESC, platform ASC
LIMIT 1
```

Returns: 1 row: platform=YouTube, n_posts=1770

E1b - verification of E1

Tests. (context for E1) The spread across the five labelled platforms is small. Report the full league table and test the gap.

```
WITH labelled AS (
    SELECT platform
    FROM posts
    WHERE platform <> 'Unspecified'
),
obs AS (
    SELECT platform, COUNT(*) AS n_posts
    FROM labelled
    GROUP BY platform
),
expectation AS (
    SELECT COUNT(*) AS total_posts,
           COUNT(DISTINCT platform) AS n_platforms
    FROM labelled
),
terms AS (
    SELECT o.platform,
           o.n_posts,
           e.total_posts * 1.0 / e.n_platforms AS expected_if_uniform,
           (o.n_posts - e.total_posts * 1.0 / e.n_platforms)
             * (o.n_posts - e.total_posts * 1.0 / e.n_platforms)
             / (e.total_posts * 1.0 / e.n_platforms) AS chi2_term
    FROM obs o CROSS JOIN expectation e
)
SELECT platform,
       n_posts,
       ROUND(100.0 * n_posts / SUM(n_posts) OVER (), 2)           AS pct_share,
       ROUND(expected_if_uniform, 1)                               AS expected_if_uniform,
       ROUND(chi2_term, 4)                                         AS chi2_term,
       ROUND(SUM(chi2_term) OVER (), 4)                           AS chi2_total,
       4                                                            AS df,
       CASE WHEN SUM(chi2_term) OVER () > 9.488
            THEN 'field is NOT uniform -- the leader is genuinely ahead'
            ELSE 'field is flat -- the leader is ahead only by noise' END AS verdict_at_5pct
FROM terms
ORDER BY n_posts DESC
```

Returns: 5 rows, led by platform=YouTube, n_posts=1770

E2 - Most Engaged Posts

Challenge. Find the top 10 posts based on total engagement. Total engagement is defined as likes + shares + comments. Ignore posts where likes are missing.

```
SELECT post_id,
       user_id,
       platform,
       likes,
       shares,
       comments,
       likes + COALESCE(shares, 0) + COALESCE(comments, 0) AS total_engagement
FROM posts
WHERE likes IS NOT NULL
ORDER BY total_engagement DESC, post_id ASC
LIMIT 10
```

Returns: 10 rows, led by post_id=ycjj5zzt7mvx, user_id=user_d9971ba6

E2b - verification of E2

Tests. (context for E2) A top-10 is only meaningful if its cut is meaningful. Quantify the cut.

```
WITH ranked AS (
  SELECT post_id,
         likes + COALESCE(shares, 0) + COALESCE(comments, 0) AS total_engagement
  FROM posts
  WHERE likes IS NOT NULL
),
cut AS (
  SELECT total_engagement AS tenth_place
  FROM ranked
  ORDER BY total_engagement DESC, post_id ASC
  LIMIT 1 OFFSET 9
)
SELECT (SELECT COUNT(*) FROM ranked) AS comparable_posts,
       (SELECT MAX(total_engagement) FROM ranked) AS best_engagement,
       cut.tenth_place AS tenth_place_engagement,
       (SELECT COUNT(*) FROM ranked, cut
        WHERE ranked.total_engagement >= 0.99 * cut.tenth_place) AS posts_within_1pct_of_cut,
       ROUND(100.0 * (SELECT MAX(total_engagement) FROM ranked)
             / cut.tenth_place - 100.0, 2) AS best_over_tenth_pct
FROM cut
```

Returns: 1 row: comparable_posts=8689, best_engagement=7893

E3 - Average Engagement By Platform

Challenge. Calculate the average likes, shares, and comments for each platform. Which platform generates the highest average total engagement?

```
SELECT platform,
       COUNT(*) AS n_posts,
       COUNT(likes) AS rows_with_likes,
       ROUND(AVG(likes), 1) AS avg_likes,
       ROUND(AVG(shares), 1) AS avg_shares,
       ROUND(AVG(comments), 1) AS avg_comments,
       ROUND(AVG(engagement), 1) AS avg_total_engagement,
       RANK() OVER (ORDER BY AVG(engagement) DESC) AS engagement_rank,
       CASE WHEN platform = 'Unspecified'
            THEN 'DATA GAP -- not a platform, not eligible to win'
            ELSE 'labelled platform' END AS eligibility
FROM v_posts_enriched
GROUP BY platform
ORDER BY avg_total_engagement DESC
```

Returns: 6 rows, led by platform=Instagram, n_posts=1689

E3b - verification of E3

Tests. (context for E3) Name the winning platform and justify it.

```
WITH plat AS (
    SELECT platform,
           COUNT(*) AS n_posts,
           AVG(engagement) AS mean_engagement,
           MAX(0, AVG(engagement * engagement)
              - AVG(engagement) * AVG(engagement)) AS var_engagement
    FROM v_posts_enriched
    WHERE platform <> 'Unspecified'
    GROUP BY platform
),
ranked AS (
    SELECT platform, n_posts, mean_engagement, var_engagement,
           ROW_NUMBER() OVER (ORDER BY mean_engagement DESC,
                                platform ASC) AS rn
    FROM plat
),
pair AS (
    SELECT w.platform AS winner,      w.mean_engagement AS win_mean,
           w.n_posts AS win_posts,    w.var_engagement AS win_var,
           r.platform AS runner_up,    r.mean_engagement AS run_mean,
           r.n_posts AS run_posts,    r.var_engagement AS run_var
    FROM ranked w
    JOIN ranked r ON r.rn = 2
    WHERE w.rn = 1
),
test AS (
    SELECT *,
           SQRT(win_var / win_posts + run_var / run_posts) AS std_error
    FROM pair
)
SELECT winner,
       ROUND(win_mean, 1) AS winner_avg_engagement,
       runner_up,
       ROUND(run_mean, 1) AS runner_up_avg_engagement,
       ROUND(win_mean - run_mean, 1) AS absolute_gap,
       ROUND(100.0 * (win_mean - run_mean) / run_mean, 2) AS gap_pct,
       ROUND((win_mean - run_mean) / std_error, 3) AS z_stat,
       CASE WHEN ABS((win_mean - run_mean) / std_error) > 1.96
            THEN 'SIGNIFICANT at 5%'
            ELSE 'NOT significant at 5% -- the ranking is a coin toss' END AS verdict
FROM test
```

Returns: 1 row: winner=Instagram, winner_avg_engagement=3,673.2

E4 - Highly Shared But Poorly Liked

Challenge. Identify posts that received more than 1,500 shares but fewer than 500 likes. Return the post ID, platform, likes, shares, and comments.

```
SELECT post_id,
       platform,
       likes,
       shares,
       comments
FROM posts
WHERE shares > 1500
      AND likes < 500
ORDER BY shares DESC, post_id ASC
```

Returns: 214 rows, led by post_id=euvr0r10wrj6, platform=Facebook

E4b - verification of E4

Tests. (context for E4) 214 posts match. Decide whether the pattern is platform-specific or spread across the whole platform.

```
WITH flagged AS (  
    SELECT post_id  
    FROM posts  
    WHERE shares > 1500 AND likes < 500  
)  
SELECT p.platform,  
    COUNT(*) AS n_posts,  
    SUM(CASE WHEN f.post_id IS NOT NULL THEN 1 ELSE 0 END) AS flagged_posts,  
    ROUND(100.0 * SUM(CASE WHEN f.post_id IS NOT NULL THEN 1 ELSE 0 END)  
        / COUNT(*), 2) AS pct_of_platform  
FROM posts p  
LEFT JOIN flagged f ON f.post_id = p.post_id  
GROUP BY p.platform  
ORDER BY pct_of_platform DESC
```

Returns: 6 rows, led by platform=Instagram, n_posts=1689

E5 - Users With Large Audiences

Challenge. Find all users with more than 40,000 followers. Display their user ID, location, language, and follower count.

```
SELECT user_id,  
    location,  
    language,  
    follower_count  
FROM users  
WHERE follower_count > 40000  
ORDER BY follower_count DESC, user_id ASC
```

Returns: 293 rows, led by user_id=user_3o7w66o2, location=Berlin, Germany

MEDIUM questions

M1 - Engagement By Location

Challenge. Using both datasets, calculate the total engagement generated by users from each location. Return the location, number of posts, and total engagement. Rank locations from highest to lowest engagement.

```
SELECT u.location,
       COUNT(*)           AS n_posts,
       SUM(p.engagement)   AS total_engagement,
       RANK() OVER (ORDER BY SUM(p.engagement) DESC) AS engagement_rank
FROM v_posts_enriched p
JOIN users u ON u.user_id = p.user_id
GROUP BY u.location
ORDER BY total_engagement DESC, u.location ASC
```

Returns: 33 rows, led by location=Los Angeles, USA, n_posts=391

M1b - verification of M1

Tests. (context for M1) Total engagement is a product of volume and quality. Separate the two.

```
WITH loc AS (
  SELECT u.location,
         COUNT(*)           AS n_posts,
         SUM(p.engagement)   AS total_engagement,
         AVG(p.engagement)   AS avg_engagement
  FROM v_posts_enriched p
  JOIN users u ON u.user_id = p.user_id
  GROUP BY u.location
),
ranked AS (
  SELECT location, n_posts, total_engagement, avg_engagement,
         RANK() OVER (ORDER BY total_engagement DESC) AS rank_by_total,
         RANK() OVER (ORDER BY avg_engagement DESC) AS rank_by_avg_per_post
  FROM loc
)
SELECT location,
       n_posts,
       total_engagement,
       ROUND(avg_engagement, 1) AS avg_engagement_per_post,
       rank_by_total,
       rank_by_avg_per_post,
       rank_by_total - rank_by_avg_per_post AS rank_shift
FROM ranked
ORDER BY rank_by_total
```

Returns: 33 rows, led by location=Los Angeles, USA, n_posts=391

M2 - Do High Follower Users Get More Engagement?

Challenge. Divide users into two groups: high follower users ($\geq 25,000$ followers) and low follower users ($< 25,000$ followers). Compare their average engagement per post.

```
WITH cohorts AS (  
  SELECT CASE WHEN u.follower_count >= 25000  
    THEN 'HIGH (>= 25k followers)'  
    ELSE 'LOW (< 25k followers)' END AS cohort,  
    COUNT(DISTINCT u.user_id) AS n_users,  
    COUNT(*) AS n_posts,  
    AVG(p.engagement) AS avg_engagement,  
    SUM(p.engagement) AS total_engagement,  
    SUM(p.engagement * p.engagement) * 1.0 / COUNT(*)  
      - AVG(p.engagement) * AVG(p.engagement) AS var_engagement  
  FROM v_posts_enriched p  
  JOIN users u ON u.user_id = p.user_id  
  GROUP BY cohort  
)  
SELECT cohort,  
  n_users,  
  n_posts,  
  ROUND(avg_engagement, 2) AS avg_engagement_per_post,  
  total_engagement,  
  ROUND(SQRT(MAX(var_engagement, 0)), 1) AS sd_engagement  
FROM cohorts  
ORDER BY avg_engagement DESC
```

Returns: 2 rows, led by cohort=LOW (< 25k followers), n_users=758

M2b - verification of M2

Tests. (context for M2) Say whether the gap in M2 is a finding.

```
WITH per_post AS (  
  SELECT CASE WHEN u.follower_count >= 25000 THEN 1 ELSE 0 END AS is_high,  
    p.engagement AS engagement  
  FROM v_posts_enriched p  
  JOIN users u ON u.user_id = p.user_id  
)  
),  
agg AS (  
  SELECT is_high,  
    COUNT(*) AS n,  
    AVG(engagement) AS mean_engagement,  
    MAX(0, AVG(engagement * engagement)  
      - AVG(engagement) * AVG(engagement)) AS var_engagement  
  FROM per_post  
  GROUP BY is_high  
)  
)  
pair AS (  
  SELECT h.mean_engagement AS high_mean, h.n AS high_n, h.var_engagement AS high_var,  
    l.mean_engagement AS low_mean, l.n AS low_n, l.var_engagement AS low_var  
  FROM agg h  
  JOIN agg l ON l.is_high = 0  
  WHERE h.is_high = 1  
)  
SELECT ROUND(low_mean, 2) AS low_follower_avg,  
  ROUND(high_mean, 2) AS high_follower_avg,  
  ROUND(high_mean - low_mean, 2) AS difference,  
  ROUND(100.0 * (high_mean - low_mean) / low_mean, 2) AS difference_pct,  
  ROUND((high_mean - low_mean)  
    / SQRT(high_var / high_n + low_var / low_n), 3) AS z_stat,  
  CASE WHEN ABS((high_mean - low_mean)  
    / SQRT(high_var / high_n + low_var / low_n)) > 1.96  
    THEN 'SIGNIFICANT at 5%'  
    ELSE 'NOT significant at 5% -- follower count does not predict engagement here' END AS verdict  
FROM pair
```

Returns: 1 row: low_follower_avg=3,647.98, high_follower_avg=3,598.82

M3 - Most Active Users

Challenge. Find the top 10 users who have created the highest number of posts. Display their follower count, location, post count, and total engagement.

```
SELECT user_id,
       follower_count,
       location,
       n_posts,
       total_engagement
FROM v_user_totals
ORDER BY n_posts DESC, total_engagement DESC, user_id ASC
LIMIT 10
```

Returns: 10 rows, led by user_id=user_nfo3ih5u, follower_count=40429

M3b - verification of M3

Tests. (context for M3) Report the activity distribution so the top-10 cut can be judged.

```
WITH counts AS (
  SELECT user_id, COUNT(*) AS n_posts
  FROM posts
  GROUP BY user_id
),
cut AS (
  SELECT n_posts AS tenth_place_posts
  FROM counts
  ORDER BY n_posts DESC, user_id ASC
  LIMIT 1 OFFSET 9
)
SELECT (SELECT MAX(n_posts) FROM counts)           AS busiest_user_posts,
       (SELECT MIN(n_posts) FROM counts)           AS quietest_user_posts,
       cut.tenth_place_posts                       AS tenth_place_posts,
       (SELECT COUNT(*) FROM counts, cut
        WHERE counts.n_posts = cut.tenth_place_posts) AS users_tied_at_the_cut,
       (SELECT COUNT(DISTINCT n_posts) FROM counts) AS distinct_post_counts
FROM cut
```

Returns: 1 row: busiest_user_posts=17, quietest_user_posts=1

M4 - Platform Behaviour By High Follower Users

Challenge. Among users with at least 30,000 followers, determine which platform gives them the highest average engagement per post.

```
WITH eligible AS (
  SELECT user_id
  FROM users
  WHERE follower_count >= 30000
)
SELECT p.platform,
       COUNT(*) AS n_posts,
       ROUND(AVG(p.engagement), 1) AS avg_engagement_per_post,
       RANK() OVER (ORDER BY AVG(p.engagement) DESC) AS rank_within_cohort
FROM v_posts_enriched p
JOIN eligible e ON e.user_id = p.user_id
WHERE p.platform <> 'Unspecified'
GROUP BY p.platform
ORDER BY avg_engagement_per_post DESC
```

Returns: 5 rows, led by platform=Instagram, n_posts=684

M4b - verification of M4

Tests. (context for M4) The 30,000 threshold is arbitrary. Sweep it.

```
WITH thresholds(follower_threshold) AS (
  VALUES (25000), (30000), (35000), (40000), (45000)
),
per_platform AS (
  SELECT t.follower_threshold,
         p.platform,
         COUNT(*)                AS n_posts,
         AVG(p.engagement)        AS avg_engagement,
         ROW_NUMBER() OVER (PARTITION BY t.follower_threshold
                             ORDER BY AVG(p.engagement) DESC, p.platform ASC) AS rn
  FROM thresholds t
  JOIN users u ON u.follower_count >= t.follower_threshold
  JOIN v_posts_enriched p ON p.user_id = u.user_id
  WHERE p.platform <> 'Unspecified'
  GROUP BY t.follower_threshold, p.platform
)
SELECT follower_threshold,
       platform                AS winning_platform,
       n_posts,
       ROUND(avg_engagement, 1) AS avg_engagement_per_post
FROM per_platform
WHERE rn = 1
ORDER BY follower_threshold
```

Returns: 5 rows, led by follower_threshold=25000, winning_platform=Instagram

M5 - Detect Suspicious Engagement

Challenge. Find posts where the number of shares is greater than the number of likes and comments combined. Such posts may represent unusual sharing behaviour. Return the top 20.

```
SELECT post_id,
       user_id,
       platform,
       likes,
       shares,
       comments,
       shares - likes - COALESCE(comments, 0)                AS excess_shares,
       ROUND(shares * 1.0 / NULLIF(likes + COALESCE(comments, 0), 0), 2)
                                                                AS share_to_reaction_ratio
FROM v_posts_enriched
WHERE likes IS NOT NULL
      AND shares > likes + COALESCE(comments, 0)
ORDER BY excess_shares DESC, post_id ASC
LIMIT 20
```

Returns: 20 rows, led by post_id=603lc32fqk41, user_id=user_bq4d19qm

M5b - verification of M5

Tests. (context for M5) Show what the alternative reading would produce.

```
WITH strict_count AS (  
    SELECT COUNT(*) AS n  
    FROM v_posts_enriched  
    WHERE likes IS NOT NULL AND shares > likes + COALESCE(comments, 0)  
)  
,  
loose_count AS (  
    SELECT COUNT(*) AS n  
    FROM v_posts_enriched  
    WHERE shares > COALESCE(likes, 0) + COALESCE(comments, 0)  
)  
,  
missing_like_count AS (  
    SELECT COUNT(*) AS n  
    FROM v_posts_enriched  
    WHERE likes IS NULL  
    AND shares > COALESCE(likes, 0) + COALESCE(comments, 0)  
)  
SELECT strict_count.n AS strict_definition,  
       loose_count.n AS likes_read_as_zero,  
       missing_like_count.n AS anomalies_created_by_blanks,  
       loose_count.n - strict_count.n AS arithmetic_gap,  
       ROUND(100.0 * missing_like_count.n / NULLIF(loose_count.n, 0), 2) AS pct_of_loose_that_is_missing_data,  
       CASE WHEN loose_count.n - strict_count.n = missing_like_count.n  
            THEN 'every extra row in the loose reading is a missing like count'  
            ELSE 'the two readings differ for a reason that is NOT missing data' END AS reconciliation  
FROM strict_count, loose_count, missing_like_count
```

Returns: 1 row: strict_definition=1047, likes_read_as_zero=2201

HARD questions

H1 - Abnormally High Engagement

Challenge. Calculate the average total engagement per user. Identify users whose average engagement per post is more than twice the overall average engagement per post. Return the user ID, location, follower count, post count, and average engagement.

```
WITH per_user AS (
    SELECT user_id,
           COUNT(*)          AS n_posts,
           AVG(engagement)   AS avg_engagement
    FROM v_posts_enriched
    GROUP BY user_id
),
baseline AS (
    SELECT AVG(engagement) AS overall_avg_per_post
    FROM v_posts_enriched
)
SELECT pu.user_id,
       u.location,
       u.follower_count,
       pu.n_posts,
       ROUND(pu.avg_engagement, 2) AS avg_engagement
FROM per_user pu
JOIN users u ON u.user_id = pu.user_id
CROSS JOIN baseline b
WHERE pu.avg_engagement > 2.0 * b.overall_avg_per_post
ORDER BY pu.avg_engagement DESC, pu.user_id ASC
```

Returns: 0 rows - the empty set is the answer

H1b - verification of H1

Tests. (context for H1) An empty result must be shown to be arithmetic, not a broken query.

```
WITH per_user AS (
    SELECT user_id,
           COUNT(*)          AS n_posts,
           AVG(engagement)   AS avg_engagement
    FROM v_posts_enriched
    GROUP BY user_id
),
baseline AS (
    SELECT AVG(engagement) AS overall_avg_per_post
    FROM v_posts_enriched
)
SELECT ROUND(b.overall_avg_per_post, 2)          AS overall_avg_per_post,
       ROUND(2.0 * b.overall_avg_per_post, 2)    AS threshold_2x,
       (SELECT COUNT(*) FROM per_user)           AS users_examined,
       (SELECT ROUND(MAX(avg_engagement), 2) FROM per_user) AS highest_user_avg,
       (SELECT user_id FROM per_user
        ORDER BY avg_engagement DESC, user_id ASC LIMIT 1) AS highest_avg_user,
       ROUND((SELECT MAX(avg_engagement) FROM per_user)
             / b.overall_avg_per_post, 3)        AS highest_avg_as_multiple_of_baseline,
       (SELECT COUNT(*) FROM per_user
        WHERE avg_engagement > 2.0 * b.overall_avg_per_post) AS qualifying_users,
       CASE WHEN (SELECT MAX(avg_engagement) FROM per_user)
              > 2.0 * b.overall_avg_per_post
        THEN 'threshold is reachable -- empty result would be a bug'
        ELSE 'EMPTY BY ARITHMETIC -- no user reaches 2x, so H1 returning 0 rows is the answer' END AS verdict
FROM baseline b
```

Returns: 1 row: overall_avg_per_post=3,623.72, threshold_2x=7,247.43

H1c - verification of H1

Tests. (context for H1) Find the multiple at which the question does have an answer, so the 2x cut can be reported against a real curve.

```
WITH per_user AS (  
    SELECT user_id,  
           AVG(engagement) AS avg_engagement  
    FROM v_posts_enriched  
    GROUP BY user_id  
,  
baseline AS (  
    SELECT AVG(engagement) AS overall_avg_per_post  
    FROM v_posts_enriched  
,  
multiples(multiple, factor, label) AS (  
    VALUES (1, 1.25, '1.25x'), (2, 1.50, '1.50x'), (3, 1.75, '1.75x'), (4, 2.00, '2.00x')  
)  
SELECT m.label AS multiple,  
       ROUND(m.factor * b.overall_avg_per_post, 2) AS threshold,  
       (SELECT COUNT(*) FROM per_user p  
        WHERE p.avg_engagement > m.factor * b.overall_avg_per_post) AS users_qualifying  
FROM multiples m  
CROSS JOIN baseline b  
ORDER BY m.multiple
```

Returns: 4 rows, led by multiple=1.25x, threshold=4,529.65

H2 - Rank Users Within Their Location

Challenge. For every location, rank users based on their total engagement. Return only the top 3 users from each location.

```
WITH ranked AS (  
    SELECT u.location,  
           v.user_id,  
           u.follower_count,  
           v.n_posts,  
           v.total_engagement,  
           ROW_NUMBER() OVER (PARTITION BY u.location  
                               ORDER BY v.total_engagement DESC,  
                                       v.n_posts DESC,  
                                       v.user_id ASC) AS rank_in_location  
    FROM v_user_totals v  
    JOIN users u ON u.user_id = v.user_id  
)  
SELECT location,  
       rank_in_location,  
       user_id,  
       follower_count,  
       n_posts,  
       total_engagement  
FROM ranked  
WHERE rank_in_location <= 3  
ORDER BY location ASC, rank_in_location ASC
```

Returns: 99 rows, led by location=Barcelona, Spain, rank_in_location=1

H2b - verification of H2

Tests. (context for H2) Show which locations have a contested 3rd place.

```
WITH ranked AS (
    SELECT u.location,
           v.total_engagement,
           ROW_NUMBER() OVER (PARTITION BY u.location
                              ORDER BY v.total_engagement DESC,
                                       v.n_posts DESC,
                                       v.user_id ASC) AS rn
    FROM v_user_totals v
    JOIN users u ON u.user_id = v.user_id
),
podium AS (
    SELECT location,
           MAX(CASE WHEN rn = 3 THEN total_engagement END) AS third_place,
           MIN(CASE WHEN rn = 4 THEN total_engagement END) AS fourth_place
    FROM ranked
    GROUP BY location
)
SELECT location,
       third_place,
       fourth_place,
       third_place - fourth_place AS absolute_margin,
       ROUND(100.0 * (third_place - fourth_place) / fourth_place, 2) AS margin_pct
FROM podium
ORDER BY margin_pct ASC, location ASC
LIMIT 8
```

Returns: 8 rows, led by location=Tokyo, Japan, third_place=43092

H3 - Platform Performance Vs Its Own Average

Challenge. For every platform, identify posts whose engagement is significantly higher than the average engagement of that platform. A post is considered exceptional if its engagement is at least 2x the average engagement of its platform.

```
WITH plat AS (
    SELECT platform,
           COUNT(*) AS platform_posts,
           AVG(engagement) AS platform_avg_engagement
    FROM v_posts_enriched
    WHERE platform <> 'Unspecified'
    GROUP BY platform
)
SELECT p.post_id,
       p.user_id,
       p.platform,
       p.likes,
       p.shares,
       p.comments,
       ROUND(pl.platform_avg_engagement, 1) AS platform_avg_engagement,
       p.engagement AS post_engagement,
       ROUND(p.engagement * 1.0 / pl.platform_avg_engagement, 3) AS multiple_of_platform_avg
FROM v_posts_enriched p
JOIN plat pl ON pl.platform = p.platform
WHERE p.engagement >= 2.0 * pl.platform_avg_engagement
ORDER BY multiple_of_platform_avg DESC, p.post_id ASC
```

Returns: 56 rows, led by post_id=wo7py9aljg3t, user_id=user_o8le7hqf

H3b - verification of H3

Tests. (context for H3) Show each platform's bar and what the excluded bucket would have contributed.

```
WITH plat AS (  
    SELECT platform,  
           COUNT(*) AS n_posts,  
           AVG(engagement) AS platform_avg_engagement  
    FROM v_posts_enriched  
    GROUP BY platform  
)  
,  
hits AS (  
    SELECT p.platform, COUNT(*) AS exceptional_posts  
    FROM v_posts_enriched p  
    JOIN plat pl ON pl.platform = p.platform  
    WHERE p.engagement >= 2.0 * pl.platform_avg_engagement  
    GROUP BY p.platform  
)  
SELECT pl.platform,  
       pl.n_posts,  
       ROUND(pl.platform_avg_engagement, 1) AS platform_avg_engagement,  
       ROUND(2.0 * pl.platform_avg_engagement, 1) AS exceptional_threshold,  
       COALESCE(h.exceptional_posts, 0) AS exceptional_posts,  
       ROUND(100.0 * COALESCE(h.exceptional_posts, 0) / pl.n_posts, 2) AS pct_of_platform,  
       CASE WHEN pl.platform = 'Unspecified'  
            THEN 'DATA GAP -- excluded from the H3 answer'  
            ELSE 'labelled platform' END AS eligibility  
FROM plat pl  
LEFT JOIN hits h ON h.platform = pl.platform  
ORDER BY pct_of_platform DESC, pl.platform ASC
```

Returns: 6 rows, led by platform=Twitter, n_posts=1742

H4 - Follower To Engagement Anomaly

Challenge. Find users who have fewer than 5,000 followers but whose total post engagement places them among the top 10% of all users. These users may represent unusually high performing or suspicious accounts. This requires multiple levels of analysis.

```
WITH user_totals AS (
  SELECT user_id,
         COUNT(*) AS n_posts,
         SUM(engagement) AS total_engagement,
         AVG(engagement) AS avg_engagement
  FROM v_posts_enriched
  GROUP BY user_id
),
bucketed AS (
  SELECT user_id,
         n_posts,
         total_engagement,
         avg_engagement,
         NTILE(10) OVER (ORDER BY total_engagement DESC) AS engagement_decile
  FROM user_totals
),
deciles AS (
  SELECT user_id,
         n_posts,
         total_engagement,
         avg_engagement,
         engagement_decile,
         MIN(total_engagement) OVER (PARTITION BY engagement_decile) AS decile_floor
  FROM bucketed
)
SELECT u.user_id,
       u.location,
       u.follower_count,
       d.n_posts,
       d.total_engagement,
       ROUND(d.avg_engagement, 2) AS avg_engagement_per_post,
       d.engagement_decile,
       d.decile_floor AS top_decile_floor_engagement
FROM deciles d
JOIN users u ON u.user_id = d.user_id
WHERE d.engagement_decile = 1
      AND u.follower_count < 5000
ORDER BY d.total_engagement DESC, u.user_id ASC
```

Returns: 17 rows, led by user_id=user_uerv85na, location=Rome, Italy

H4b - verification of H4

Tests. (context for H4) Two independent definitions of "top 10%" must agree before the finding is reported.

```
WITH user_totals AS (
  SELECT user_id,
         SUM(engagement) AS total_engagement
  FROM v_posts_enriched
  GROUP BY user_id
),
ntile_cut AS (
  SELECT user_id,
         NTILE(10) OVER (ORDER BY total_engagement DESC) AS engagement_decile
  FROM user_totals
),
percentile_cut AS (
  SELECT user_id,
         total_engagement,
         ROW_NUMBER() OVER (ORDER BY total_engagement DESC, user_id ASC) AS rn,
         COUNT(*) OVER () AS n_users
  FROM user_totals
),
floor_value AS (
  SELECT MIN(total_engagement) AS decile_floor
  FROM percentile_cut
  WHERE rn <= n_users * 0.10
)
SELECT (SELECT decile_floor FROM floor_value)          AS top_decile_floor_engagement,
       (SELECT COUNT(*) FROM ntile_cut n
        JOIN users u ON u.user_id = n.user_id
        WHERE n.engagement_decile = 1 AND u.follower_count < 5000) AS via_ntile_10_buckets,
       (SELECT COUNT(*) FROM percentile_cut p
        JOIN users u ON u.user_id = p.user_id
        WHERE p.rn <= p.n_users * 0.10 AND u.follower_count < 5000) AS via_explicit_percentile,
       CASE WHEN (SELECT COUNT(*) FROM ntile_cut n
                  JOIN users u ON u.user_id = n.user_id
                  WHERE n.engagement_decile = 1 AND u.follower_count < 5000)
              = (SELECT COUNT(*) FROM percentile_cut p
                  JOIN users u ON u.user_id = p.user_id
                  WHERE p.rn <= p.n_users * 0.10 AND u.follower_count < 5000)
              THEN 'the two definitions AGREE -- the answer is not an artefact of NTILE'
              ELSE 'the definitions DISAGREE -- report the boundary as unstable' END AS robustness
```

Returns: 1 row: top_decile_floor_engagement=38765, via_ntile_10_buckets=17

H5 - Identify Data Anomalies

Challenge. Identify potentially corrupted posts where one or more of the following conditions are true: (1) likes are negative; (2) platform is missing; (3) text content is missing; (4) text contains HTML entities/tags such as &, <div>, or
. Return the post ID and identify the type of anomaly.

```
WITH flags AS (
  SELECT post_id,
         MAX(CASE WHEN anomaly_type = 'NEGATIVE_LIKES'      THEN 1 ELSE 0 END) AS neg_likes,
         MAX(CASE WHEN anomaly_type = 'MISSING_PLATFORM'    THEN 1 ELSE 0 END) AS no_platform,
         MAX(CASE WHEN anomaly_type = 'MISSING_TEXT'        THEN 1 ELSE 0 END) AS no_text,
         MAX(CASE WHEN anomaly_type = 'HTML_ENTITY_OR_TAG'  THEN 1 ELSE 0 END) AS html,
         COUNT(*) AS n_anomalies
  FROM raw_post_anomalies
  GROUP BY post_id
)
SELECT post_id,
       n_anomalies,
       TRIM(CASE WHEN neg_likes = 1 THEN 'NEGATIVE_LIKES ' ELSE '' END ||
            CASE WHEN no_platform = 1 THEN 'MISSING_PLATFORM ' ELSE '' END ||
            CASE WHEN no_text = 1 THEN 'MISSING_TEXT ' ELSE '' END ||
            CASE WHEN html = 1 THEN 'HTML_ENTITY_OR_TAG' ELSE '' END) AS anomaly_types
FROM flags
ORDER BY n_anomalies DESC, post_id ASC
```

Returns: 4418 rows, led by post_id=0oqpg2rgnqxa, n_anomalies=3

H5b - verification of H5

Tests. (context for H5) Reconcile the SQL scan against the Phase-1 corruption inventory.

```
WITH families(anomaly_type) AS (
  VALUES ('NEGATIVE_LIKES'), ('MISSING_PLATFORM'), ('MISSING_TEXT'), ('HTML_ENTITY_OR_TAG')
),
intake AS (
  SELECT 'NEGATIVE_LIKES' AS anomaly_type, COUNT(*) AS intake_rows
    FROM v_raw_anomaly_scan WHERE is_negative_likes = 1
  UNION ALL
  SELECT 'MISSING_PLATFORM', COUNT(*) FROM v_raw_anomaly_scan WHERE is_missing_platform = 1
  UNION ALL
  SELECT 'MISSING_TEXT', COUNT(*) FROM v_raw_anomaly_scan WHERE is_missing_text = 1
  UNION ALL
  SELECT 'HTML_ENTITY_OR_TAG', COUNT(*) FROM v_raw_anomaly_scan WHERE is_html_markup = 1
),
distinct_posts AS (
  SELECT anomaly_type, COUNT(*) AS n_posts
    FROM raw_post_anomalies
  GROUP BY anomaly_type
)
SELECT f.anomaly_type,
       i.intake_rows,
       COALESCE(d.n_posts, 0) AS distinct_posts,
       i.intake_rows - COALESCE(d.n_posts, 0) AS replay_rows_removed
FROM families f
JOIN intake i ON i.anomaly_type = f.anomaly_type
LEFT JOIN distinct_posts d ON d.anomaly_type = f.anomaly_type
ORDER BY i.intake_rows DESC
```

Returns: 4 rows, led by anomaly_type=MISSING_PLATFORM, intake_rows=1846

H5c - verification of H5

Tests. (context for H5) Rank the worst-affected posts.

```
WITH flags AS (
  SELECT post_id, COUNT(*) AS n_anomalies
    FROM raw_post_anomalies
  GROUP BY post_id
)
SELECT f.post_id,
       f.n_anomalies,
       TRIM(CASE WHEN SUM(CASE WHEN a.anomaly_type = 'NEGATIVE_LIKES' THEN 1 ELSE 0 END) = 1 THEN 'NEGATIVE_LIKES'
              ' ELSE '' END ||
              CASE WHEN SUM(CASE WHEN a.anomaly_type = 'MISSING_PLATFORM' THEN 1 ELSE 0 END) = 1 THEN
              'MISSING_PLATFORM ' ELSE '' END ||
              CASE WHEN SUM(CASE WHEN a.anomaly_type = 'MISSING_TEXT' THEN 1 ELSE 0 END) = 1 THEN 'MISSING_TEXT '
              ELSE '' END ||
              CASE WHEN SUM(CASE WHEN a.anomaly_type = 'HTML_ENTITY_OR_TAG' THEN 1 ELSE 0 END) = 1 THEN
              'HTML_ENTITY_OR_TAG' ELSE '' END) AS anomaly_types
FROM flags f
JOIN raw_post_anomalies a ON a.post_id = f.post_id
WHERE f.n_anomalies = (SELECT MAX(n_anomalies) FROM flags)
GROUP BY f.post_id, f.n_anomalies
ORDER BY f.post_id ASC
```

Returns: 20 rows, led by post_id=0oqpg2rgnqxa, n_anomalies=3

H6 - Most Suspicious High Impact Users

Challenge. Identify users who satisfy all three conditions: have fewer than 10,000 followers; their average post engagement is above the overall average; and at least one of their posts has more shares than likes. Rank these users by total engagement and display their location, follower count, number of posts, average engagement, and total engagement.

```
WITH baseline AS (
    SELECT AVG(engagement) AS overall_avg_per_post
    FROM v_posts_enriched
),
per_user AS (
    SELECT user_id,
           COUNT(*) AS n_posts,
           AVG(engagement) AS avg_engagement,
           SUM(engagement) AS total_engagement
    FROM v_posts_enriched
    GROUP BY user_id
),
over_shared AS (
    SELECT DISTINCT user_id
    FROM v_posts_enriched
    WHERE likes IS NOT NULL
       AND shares > likes
)
SELECT u.location,
       u.follower_count,
       pu.n_posts,
       ROUND(pu.avg_engagement, 2) AS avg_engagement_per_post,
       pu.total_engagement,
       RANK() OVER (ORDER BY pu.total_engagement DESC) AS engagement_rank,
       pu.user_id
FROM per_user pu
JOIN users u ON u.user_id = pu.user_id
CROSS JOIN baseline b
WHERE u.follower_count < 10000
      AND pu.avg_engagement > b.overall_avg_per_post
      AND pu.user_id IN (SELECT user_id FROM over_shared)
ORDER BY pu.total_engagement DESC, pu.user_id ASC
```

Returns: 82 rows, led by location=Johannesburg, South Africa, follower_count=8262

H6b - verification of H6

Tests. (context for H6) Show how many users each condition removes, so the answer can be seen to be selective rather than accidental.

```
WITH baseline AS (
  SELECT AVG(engagement) AS overall_avg_per_post
  FROM v_posts_enriched
),
per_user AS (
  SELECT user_id,
         AVG(engagement) AS avg_engagement,
         SUM(engagement) AS total_engagement
  FROM v_posts_enriched
  GROUP BY user_id
),
over_shared AS (
  SELECT DISTINCT user_id
  FROM v_posts_enriched
  WHERE likes IS NOT NULL AND shares > likes
)
SELECT (SELECT COUNT(*) FROM users) AS all_users,
       (SELECT COUNT(*) FROM users WHERE follower_count < 10000) AS c1_fewer_than_10k_followers,
       (SELECT COUNT(*) FROM per_user, baseline b
        WHERE avg_engagement > b.overall_avg_per_post) AS c2_above_overall_avg,
       (SELECT COUNT(*) FROM over_shared) AS c3_has_over_shared_post,
       (SELECT COUNT(*) FROM per_user pu
        JOIN users u ON u.user_id = pu.user_id, baseline b
        WHERE u.follower_count < 10000
        AND pu.avg_engagement > b.overall_avg_per_post
        AND pu.user_id IN (SELECT user_id FROM over_shared)) AS all_three_conditions
```

Returns: 1 row: all_users=1500, c1_fewer_than_10k_followers=287

Appendix - the twelve core queries (Q1-Q12)

Provided for completeness. These are the analytical core built during the data-recovery phase, in the same database, and they are not part of the E/M/H challenge set above.

Q1 - Trend Detection

Question. monthly volume, momentum, and how anomalous each

```
WITH monthly AS (
    SELECT * FROM v_monthly
),
stats AS (
    SELECT AVG(posts)          AS mu_posts,
           SQRT(AVG(posts * posts) - AVG(posts) * AVG(posts)) AS sd_posts
    FROM monthly
)
SELECT m.post_month,
       m.posts,
       LAG(m.posts) OVER (ORDER BY m.post_month)          AS prev_month,
       ROUND(100.0 * (m.posts - LAG(m.posts) OVER (ORDER BY m.post_month))
             / NULLIF(LAG(m.posts) OVER (ORDER BY m.post_month), 0), 2) AS mom_pct,
       ROUND(m.mean_engagement, 1)                          AS mean_engagement,
       ROUND((m.posts - s.mu_posts) / NULLIF(s.sd_posts, 0), 2) AS z_volume,
       RANK() OVER (ORDER BY m.posts DESC)                  AS volume_rank
FROM monthly m CROSS JOIN stats s
ORDER BY m.post_month
```

Returns: 12 rows, led by post_month=2024-05, posts=878

Q2 - Trend Detection

Question. 3-month centred moving average and the inflection

```
WITH ordered AS (
    SELECT post_month, posts,
           AVG(posts) OVER (
               ORDER BY post_month
               ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING) AS ma3,
           ROW_NUMBER() OVER (ORDER BY post_month)        AS mo
    FROM v_monthly
),
turned AS (
    SELECT o.*,
           SIGN(ma3 - LAG(ma3) OVER (ORDER BY mo)) AS prev_dir,
           SIGN(LEAD(ma3) OVER (ORDER BY mo) - ma3) AS next_dir
    FROM ordered o
)
SELECT post_month, posts, ROUND(ma3, 1) AS ma3,
       CASE WHEN prev_dir = 1 AND next_dir = -1 THEN 'PEAK'
            WHEN prev_dir = -1 AND next_dir = 1 THEN 'TROUGH'
            ELSE '-' END AS inflection
FROM turned
ORDER BY post_month
```

Returns: 12 rows, led by post_month=2024-05, posts=878

Q3 - Behavioural Grouping

Question. platform league table with the intake gap

```
SELECT platform,
       COUNT(*)
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*) OVER ()), 2)
       SUM(CASE WHEN likes IS NULL THEN 1 ELSE 0 END)
       ROUND(100.0 * SUM(CASE WHEN likes IS NULL THEN 1 ELSE 0 END)
             / COUNT(*), 2)
       ROUND(AVG(engagement), 1)
       ROUND(AVG(shares), 1)
       ROUND(AVG(comments), 1)
       CASE WHEN platform = 'Unspecified' THEN 0 ELSE 1 END
FROM v_posts_enriched
GROUP BY platform
ORDER BY posts DESC
```

Returns: 6 rows, led by platform=YouTube, posts=1770

Q4 - Anomaly Discovery

Question. amplification anomalies -- posts that are shared

```
WITH flagged AS (
  SELECT post_id, user_id, platform, post_month, likes, shares, comments,
         amplify_ratio
  FROM v_posts_enriched
  WHERE likes > 0 AND amplify_ratio > 3
)
SELECT COUNT(*)
      (SELECT COUNT(*) FROM v_posts_enriched
       WHERE likes > 0)
      ROUND(100.0 * COUNT(*) /
            (SELECT COUNT(*) FROM v_posts_enriched WHERE likes > 0), 2)
      ROUND(AVG(amplify_ratio), 2)
      ROUND(MAX(amplify_ratio), 1)
FROM flagged
```

Returns: 1 row: anomalous_posts=846, comparable_posts=8688

Q5 - Anomaly Discovery

Question. attribution -- is the amplification anomaly a

```
WITH per_user AS (
  SELECT u.user_id,
         COALESCE(a.anom, 0) AS anom
  FROM users u
  LEFT JOIN (SELECT user_id, COUNT(*) AS anom
             FROM v_posts_enriched
             WHERE likes > 0 AND amplify_ratio > 3
             GROUP BY user_id) a ON a.user_id = u.user_id
),
params AS (
  SELECT COUNT(*) AS n_users,
         SUM(anom) AS n_anom,
         SUM(anom) * 1.0 / COUNT(*) AS lambda_
  FROM per_user
),
observed AS (
  SELECT SUM(anom >= 3) AS obs_ge3,
         SUM(anom >= 4) AS obs_ge4,
         MAX(anom) AS obs_max
  FROM per_user
),
expected AS (
  SELECT n_users, lambda_,
         n_users * EXP(-1.0 * lambda_) * (1.0 + lambda_
         + POW(lambda_, 2) / 2.0) AS cum_le2,
         n_users * EXP(-1.0 * lambda_) * (1.0 + lambda_
         + POW(lambda_, 2) / 2.0 + POW(lambda_, 3) / 6.0) AS cum_le3
  FROM params
)
SELECT p.n_users,
       p.n_anom,
       ROUND(p.lambda_, 4) AS poisson_lambda,
       o.obs_ge3,
       ROUND(p.n_users - e.cum_le2, 1) AS expected_ge3_by_chance,
       o.obs_ge4,
       ROUND(p.n_users - e.cum_le3, 1) AS expected_ge4_by_chance,
       o.obs_max AS busiest_author,
       ROUND(o.obs_ge3 * 1.0 / NULLIF(p.n_users - e.cum_le2, 0), 3) AS observed_over_expected,
       CASE WHEN o.obs_ge3 BETWEEN 0.8 * (p.n_users - e.cum_le2)
               AND 1.25 * (p.n_users - e.cum_le2)
             THEN 'DIFFUSE -- indistinguishable from random scatter, NOT a coordinated ring'
             ELSE 'CONCENTRATED -- attribution is warranted' END AS verdict
FROM params p CROSS JOIN observed o CROSS JOIN expected e
```

Returns: 1 row: n_users=1500, n_anom=846

Q6 - Data-Quality Forensics

Question. the midnight illusion -- all 24 hours

```
SELECT h.post_hour,
       COUNT(*) AS rows_naive,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*) OVER ()), 2) AS pct_naive,
       SUM(h.has_time) AS rows_with_real_time,
       ROUND(100.0 * SUM(h.has_time) / SUM(SUM(h.has_time) OVER ()), 2) AS pct_valid,
       COUNT(*) - SUM(h.has_time) AS artefact_rows
FROM v_posts_enriched h
GROUP BY h.post_hour
ORDER BY artefact_rows DESC, h.post_hour
```

Returns: 24 rows, led by post_hour=0, rows_naive=3325

Q7 - Behavioural Grouping

Question. RFM segmentation of the author base

```
WITH base AS (
    SELECT user_id,
           CAST(JULIANDAY((SELECT MAX(posted_at) FROM posts))
                - JULIANDAY(MAX(posted_at))) AS INTEGER) AS recency_days,
           COUNT(*) AS frequency,
           SUM(engagement) AS monetary
    FROM v_posts_enriched
    GROUP BY user_id
),
scored AS (
    SELECT b.*,
           NTILE(5) OVER (ORDER BY recency_days DESC) AS r_tile, -- small days = recent
           NTILE(5) OVER (ORDER BY frequency) AS f_tile,
           NTILE(5) OVER (ORDER BY monetary) AS m_tile
    FROM base b
)
SELECT CASE
    WHEN r_tile >= 4 AND f_tile >= 4 AND m_tile >= 4 THEN 'champions'
    WHEN r_tile >= 4 AND f_tile <= 2 THEN 'new_and_promising'
    WHEN r_tile <= 2 AND f_tile >= 4 THEN 'drifting_regulars'
    WHEN r_tile <= 2 AND m_tile <= 2 THEN 'lapsed_low_value'
    ELSE 'mid_value'
END AS segment,
COUNT(*) AS users,
ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS share_pct,
ROUND(AVG(frequency), 2) AS avg_posts,
ROUND(AVG(monetary), 1) AS avg_total_engagement,
ROUND(AVG(recency_days), 0) AS avg_days_since_last_post
FROM scored
GROUP BY segment
ORDER BY users DESC
```

Returns: 5 rows, led by segment=mid_value, users=592

Q8 - Correlation Analysis

Question. does follower count buy engagement?

```
WITH per_user AS (
    SELECT p.user_id,
           AVG(p.engagement) AS mean_engagement,
           u.follower_count
    FROM v_posts_enriched p
    JOIN users u ON u.user_id = p.user_id
    GROUP BY p.user_id, u.follower_count
),
agg AS (
    SELECT COUNT(*) AS n,
           SUM(follower_count) AS sx,
           SUM(mean_engagement) AS sy,
           SUM(follower_count * mean_engagement) AS sxy,
           SUM(follower_count * follower_count) AS sxx,
           SUM(mean_engagement * mean_engagement) AS syy
    FROM per_user
)
SELECT n AS users,
       ROUND((n*sxy - sx*sy) /
             (SQRT((n*sxx - sx*sx) * (n*syy - sy*sy))), 4) AS pearson_r,
       ROUND(((n*sxy - sx*sy) / (SQRT((n*sxx - sx*sx) * (n*syy - sy*sy))))
             * ((n*sxy - sx*sy) / (SQRT((n*sxx - sx*sx) * (n*syy - sy*sy))))
             * 100, 2) AS r_squared_pct,
       CASE WHEN ABS((n*sxy - sx*sy) /
                     (SQRT((n*sxx - sx*sx) * (n*syy - sy*sy)))) < 0.1
            THEN 'no material relationship'
            ELSE 'relationship worth modelling' END AS verdict
FROM agg
```

Returns: 1 row: users=1500, pearson_r=-0.04

Q9 - Correlation Analysis

Question. same question, bucketed -- because a near-zero

```
WITH ranked AS (  
    SELECT u.follower_count,  
           NTILE(5) OVER (ORDER BY u.follower_count) AS quintile,  
           p.engagement  
    FROM users u JOIN v_posts_enriched p ON p.user_id = u.user_id  
)  
SELECT quintile,  
       CASE quintile WHEN 1 THEN '1 (fewest)' WHEN 2 THEN '2' WHEN 3 THEN '3'  
                   WHEN 4 THEN '4' ELSE '5 (most)' END AS follower_band,  
       MIN(follower_count) AS min_followers,  
       MAX(follower_count) AS max_followers,  
       COUNT(*)           AS posts,  
       ROUND(AVG(engagement), 1) AS mean_engagement  
FROM ranked  
GROUP BY quintile  
ORDER BY quintile
```

Returns: 5 rows, led by quintile=1, follower_band=1 (fewest)

Q10 - Trend + Grouping

Question. hashtag momentum -- which tag accelerated fastest

```
WITH tag_month AS (  
    SELECT pt.tag, p.post_month,  
           COUNT(DISTINCT p.post_id) AS posts  
    FROM post_tags pt JOIN posts p ON p.post_id = pt.post_id  
    GROUP BY pt.tag, p.post_month  
)  
share AS (  
    SELECT tm.*,  
           ROUND(100.0 * tm.posts / m.month_posts, 2) AS pct_of_month,  
           m.month_posts  
    FROM tag_month tm  
    JOIN (SELECT post_month, COUNT(*) AS month_posts FROM posts GROUP BY post_month) m  
        ON m.post_month = tm.post_month  
)  
pivoted AS (  
    SELECT s.*,  
           LAG(pct_of_month) OVER (PARTITION BY tag ORDER BY post_month) AS prev_pct  
    FROM share s  
)  
SELECT tag,  
       MAX(CASE WHEN post_month = (SELECT MAX(post_month) FROM posts) THEN pct_of_month END) AS pct_final_month,  
       MAX(CASE WHEN post_month = (SELECT MIN(post_month) FROM posts) THEN pct_of_month END) AS pct_first_month,  
       ROUND(MAX(CASE WHEN post_month = (SELECT MAX(post_month) FROM posts) THEN pct_of_month END)  
             - MAX(CASE WHEN post_month = (SELECT MIN(post_month) FROM posts) THEN pct_of_month END), 2)  
       AS pp_change,  
       MAX(pct_of_month - prev_pct) AS best_single_month_gain_pp  
FROM pivoted  
GROUP BY tag  
ORDER BY pp_change DESC  
LIMIT 8
```

Returns: 8 rows, led by tag=Tech, pct_final_month=8.18

Q11 - Anomaly Discovery (Gap & Islands)

Question. posting streaks per user

```
WITH days AS (
    SELECT user_id, post_date,
           CAST(JULIANDAY(post_date) AS INTEGER) AS d
    FROM posts
    GROUP BY user_id, post_date
),
grp AS (
    SELECT user_id, d,
           d - ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY d) AS island
    FROM days
)
SELECT streak_len,
       COUNT(*) AS streaks,
       ROUND(AVG(posts_in_streak),1) AS avg_posts_per_day,
       MAX(posts_in_streak) AS max_posts_in_one_day
FROM (
    SELECT user_id, island,
           COUNT(*) AS posts_in_streak,
           COUNT(*) AS streak_len
    FROM grp GROUP BY user_id, island
)
GROUP BY streak_len
ORDER BY streak_len
```

Returns: 3 rows, led by streak_len=1, streaks=9763

Q12 - Integrity Gate

Question. prove the restored tables can answer anything above

```
SELECT 'duplicate post_id' AS check_name,
       COUNT(*) - COUNT(DISTINCT post_id) AS violations FROM posts
UNION ALL
SELECT 'orphan user_id',
       (SELECT COUNT(*) FROM posts p
        WHERE NOT EXISTS (SELECT 1 FROM users u WHERE u.user_id = p.user_id))
UNION ALL
SELECT 'negative engagement',
       (SELECT COUNT(*) FROM posts
        WHERE COALESCE(likes,0) < 0 OR COALESCE(shares,0) < 0
        OR COALESCE(comments,0) < 0)
UNION ALL
SELECT 'post before signup',
       (SELECT COUNT(*) FROM posts p JOIN users u USING (user_id)
        WHERE p.posted_at < u.account_created)
UNION ALL
SELECT 'timestamps outside window',
       (SELECT COUNT(*) FROM posts
        WHERE posted_at < '2024-05-01' OR posted_at >= '2025-05-01')
```

Returns: 5 rows, led by check_name=duplicate post_id, violations=0